
MAZE ROBOT

Technical Report — STM32F446RE Version

STM32F446RET6 · HC-SR04 · MPU-6050 · BTS7960B · Flood Fill

Maze Robot Team

Development board: NUCLEO-F446RE

Toolchain: STM32CubeIDE · HAL · GCC

Version 1.0 — May 2026

Executive Summary

This report describes the complete implementation of an autonomous maze-solving robot based on the **STM32F446RETx** microcontroller (ARM Cortex-M4 @ 168 MHz). The system integrates three HC-SR04 ultrasonic sensors, an MPU-6050 IMU via I2C, two BTS7960B motor drivers, and two quadrature encoders. The navigation algorithm is based on a **dynamic Flood Fill** (BFS) with dual closed-loop control (encoders + gyroscope).

Contents

1	Project Overview	7
1.1	Context and Objectives	7
1.1.1	Technical Objectives	7
1.2	Comparison with the ESP32 Version	8
2	Mechanical Design	9
2.1	Structure Overview	9
2.1.1	Mechanical Characteristics	9
2.2	Differential Drive System	9
3	Electrical Architecture	10
3.1	Overview	10
3.2	Electronic Components Summary	10
3.3	Voltage Dividers for HC-SR04	11
4	STM32CubeMX Configuration	12
4.1	Target Microcontroller	12
4.2	Clock Configuration (RCC)	12
4.3	GPIO Pin Assignment	14
4.4	Timer Configuration	14
4.4.1	TIM1 — PWM Generation (motor control)	14
4.4.2	TIM2 — Quadrature encoder (left motor)	15
4.4.3	TIM3 — Quadrature encoder (right motor)	15
4.4.4	TIM5 — μ s time measurement (HC-SR04)	16
4.5	I2C1 Configuration — MPU-6050	16
4.6	USART2 Configuration	16

5	Software Architecture	17
5.1	STM32CubeIDE Project Structure	17
5.2	Recommended Software Modules	18
5.3	Main Flow Diagram	20
6	Hardware Drivers (HAL STM32)	21
6.1	HC-SR04 Driver — Ultrasonic Measurement	21
6.1.1	Operating Principle	21
6.1.2	HAL Code — Distance Measurement	21
6.1.3	HC-SR04 Instances	23
6.2	BTS7960B Driver — PWM Motor Control	23
6.2.1	Principle	23
6.2.2	HAL Code — PWM Control	23
6.3	MPU-6050 Driver — I2C Gyroscope	24
6.3.1	Reading the Z Axis	24
6.4	Quadrature Encoder Reading	25
7	Navigation Algorithm — Flood Fill	27
7.1	Dynamic Flood Fill Principle	27
7.2	Map Representation	27
7.3	BFS Implementation	28
7.4	Wall Update	28
8	Control Loop and Trajectory Correction	30
8.1	Dual Closed-Loop Control	30
8.2	Gyroscopic Turn — 3 Phases	30
8.3	Configuration Parameters	31
9	Commissioning Procedure and Tests	32
9.1	Commissioning Order	32
9.2	Troubleshooting Guide	33
10	Linker Scripts and Memory	34
10.1	Memory Map	34
11	Conclusion	35

11.1 Technical Summary	35
11.2 Future Improvements	35

List of Figures

5.1	Flow diagram — main navigation loop	20
-----	---	----

List of Tables

1.1	Comparison STM32F446RE vs ESP32-WROOM-32D	8
3.1	Electronic components of the STM32 Maze Robot	10
4.1	Characteristics of the STM32F446RETx	12
4.2	Clock tree — STM32F446RE	13
4.3	Pin assignment table — maze_test.ioc	14
4.4	TIM1 parameters — Motor PWM	15
4.5	TIM2 parameters — Left encoder	15
4.6	TIM3 parameters — Right encoder	15
4.7	TIM5 parameters — μ s counter	16
4.8	I2C1 parameters — MPU-6050	16
4.9	USART2 parameters	16
5.1	Modular organization of the STM32 firmware	18
8.1	Correction levels in move_forward()	30
8.2	Critical parameters — config.h	31
9.1	Complete commissioning procedure	32
9.2	Troubleshooting table	33
10.1	Memory map of the STM32F446RETx	34

Chapter 1

Project Overview

1.1 Context and Objectives

The **Maze Robot STM32** is an autonomous mobile robot designed to explore and solve an unknown maze. It is built around the **NUCLEO-F446RE** development board featuring the **STM32F446RET6** microcontroller, an ARM Cortex-M4 core at 168 MHz with hardware FPU.

1.1.1 Technical Objectives

- ✔ Autonomous resolution of a configurable-size maze
- ✔ Real-time obstacle detection via 3 HC-SR04 sensors (front, left, right)
- ✔ Dual trajectory correction: quadrature encoders + MPU-6050 gyroscope
- ✔ High-resolution PWM generation via Timer TIM1 (168 MHz APB2)
- ✔ Debug communication via USART2 (PA2/PA3) @ 115200 baud
- ✔ Precise time measurement interface via TIM5 (μs)

1.2 Comparison with the ESP32 Version

Table 1.1: Comparison STM32F446RE vs ESP32-WROOM-32D

Criterion	STM32F446RE	ESP32-WROOM-32D
Core	ARM Cortex-M4 @ 168 MHz	Xtensa LX6 Dual-Core @ 240 MHz
FPU	Yes (hardware)	Yes
RAM	128 kB SRAM	520 kB SRAM
Flash	512 kB	4 MB
Advanced timers	TIM1 (center-aligned, dead-time)	LEDC (software PWM)
Encoders	TIM2 + TIM3 (hardware QEI)	Software interrupts
ToF/US sensors	HC-SR04 (GPIO + TIM5 μ s)	VL53L0X I2C (ToF)
Connectivity	USART/SWD	Wi-Fi + Bluetooth
Toolchain	STM32CubeIDE / HAL	Arduino IDE / PlatformIO
Debugging	SWD/JTAG hardware	Software UART

Chapter 2

Mechanical Design

2.1 Structure Overview

The mechanical structure was designed in **SolidWorks** using a **decagonal** prismatic geometry (10 faces), providing an optimized internal volume for electronics while offering multiple flat surfaces for sensor integration.

2.1.1 Mechanical Characteristics

- **Chassis:** laser-cut panels (MDF or acrylic), finger joint assembly system for self-alignment and structural rigidity
- **Propulsion:** differential drive with high-grip wheels mounted on a central axle
- **Sensor cutouts:** dedicated slots on multiple faces for HC-SR04 ultrasonic sensors, enabling 270° environmental mapping
- **Internal structure:** multi-level layout with standoffs and mounting plates to organize electronic boards

2.2 Differential Drive System

The robot uses a **differential drive** system with 2 drive wheels. Turns are achieved by differential speed between the left and right wheels, without any physical steering mechanism. This design provides maximum maneuverability (on-the-spot rotation) and a simple software architecture.

Chapter 3

Electrical Architecture

3.1 Overview

The electrical architecture is centered around the **STM32F446RET6**, leveraging its high-performance ARM Cortex-M4 core to handle real-time sensor processing and motor control.

3.2 Electronic Components Summary

Table 3.1: Electronic components of the STM32 Maze Robot

System	Component	Role	Interface
Control	STM32F446RETx	Main microcontroller	PWM, GPIO, I2C, UART
Actuation	2× BTS7960B	H-Bridge motor drivers	PWM + GPIO (enable)
Propulsion	2× DC Motor	Left + right wheel	H-Bridge
Odometry	2× QEI Encoder	Displacement counting	TIM2, TIM3 (QEI)
Obstacle	3× HC-SR04	Front/left/right detection	GPIO + TIM5 (µs)
IMU	MPU-6050	Gyroscope + accelerometer	I2C1 (PB8/PB9)
Protection	Resistive dividers	Level-shift Echo	Passive (1k+2k) 5V→3.3V
Power supply	Regulators	12V→5V, 5V→3.3V	DC-DC / LDO
Debug	USART2	Serial PC link	PA2 (TX), PA3 (RX)

3.3 Voltage Dividers for HC-SR04

Warning

The Echo pins of the HC-SR04 output a **5 V logic** signal, which is incompatible with the STM32F446 GPIOs that tolerate a maximum of **3.3 V**. A resistive voltage divider is required for each Echo signal.

The resistive divider used consists of $R_1 = 1\text{ k}\Omega$ (series) and $R_2 = 2\text{ k}\Omega$ (to GND), producing:

$$V_{\text{GPIO}} = V_{\text{Echo}} \times \frac{R_2}{R_1 + R_2} = 5\text{ V} \times \frac{2}{3} \approx 3.33\text{ V} \leq 3.6\text{ V} \quad (3.1)$$

According to the project schematic, the networks use $1\text{ k}\Omega$ values on the Echo lines with a GND reference, ensuring GPIO input protection.

Chapter 4

STM32CubeMX Configuration

4.1 Target Microcontroller

Table 4.1: Characteristics of the STM32F446RETx

Parameter	Value
Part number	STM32F446RET6
Family	STM32F4 (ARM Cortex-M4)
Package	LQFP64
Max frequency	180 MHz
SRAM	128 kB
Flash	512 kB
Advanced timers	TIM1 (4 channels, dead-time)
GP timers	TIM2, TIM3, TIM5
I2C	I2C1 (Fast mode 400 kHz)
USART	USART2 (asynchronous)
ADC	12 bits, 3 ADCs
FPU	Yes (Cortex-M4 FPU)
Dev board	NUCLEO-F446RE

4.2 Clock Configuration (RCC)

The `maze_test.ioc` file configures the clock system as follows:

Table 4.2: Clock tree — STM32F446RE

Parameter	Value
PLL source	HSE (external oscillator 8 MHz)
PLLM	8
PLLN	336
SYSCLK	168 MHz
HCLK (AHB)	168 MHz
APB1 (TIM2, TIM3, TIM5)	42 MHz → Timer clock 84 MHz
APB2 (TIM1)	84 MHz → Timer clock 168 MHz
FLASH latency	5 cycles (FLASH_LATENCY_5)

i Note

The TIM5 prescaler is set to **167** ($PRE+1 = 168$), which with $HCLK = 168\text{ MHz}$ gives a resolution of **1 μs** per tick, ideal for measuring the time-of-flight of HC-SR04 sensors.

4.3 GPIO Pin Assignment

Table 4.3: Pin assignment table — maze_test.ioc

Pin	Label	Mode	Description
PA0	—	TIM2_CH1 (Encoder)	Left encoder phase A
PA1	—	TIM2_CH2 (Encoder)	Left encoder phase B
PA2	USART_TX	USART2_TX	Serial transmission (debug)
PA3	USART_RX	USART2_RX	Serial reception
PA5	LD2 [Green Led]	GPIO_Output	Green LED (onboard)
PA6	—	TIM3_CH1 (Encoder)	Right encoder phase A
PA7	—	TIM3_CH2 (Encoder)	Right encoder phase B
PA8	—	TIM1_CH1 (PWM)	Motor PWM — channel 1
PA9	—	TIM1_CH2 (PWM)	Motor PWM — channel 2
PA10	—	TIM1_CH3 (PWM)	Motor PWM — channel 3
PA11	—	TIM1_CH4 (PWM)	Motor PWM — channel 4
PA13	TMS	SYS_JTMS-SWDIO	SWD debug (data)
PA14	TCK	SYS_JTCK-SWCLK	SWD debug (clock)
PB0	TRIG_FRONT	GPIO_Output	HC-SR04 front trigger
PB1	ECHO_FRONT	GPIO_Input	HC-SR04 front echo
PB2	TRIG_LEFT	GPIO_Output	HC-SR04 left trigger
PB3	SWO	SYS_JTDO-SWO	SWO debug
PB8	—	I2C1_SCL	I2C clock (MPU-6050)
PB9	—	I2C1_SDA	I2C data (MPU-6050)
PB10	ECHO_LEFT	GPIO_Input	HC-SR04 left echo
PB12	TRIG_RIGHT	GPIO_Output	HC-SR04 right trigger
PB13	ECHO_RIGHT	GPIO_Input	HC-SR04 right echo
PC13	—	GPIO_Input	User button (B1)
PC14	—	RCC_OSC32_IN	LSE 32 kHz oscillator
PC15	—	RCC_OSC32_OUT	LSE 32 kHz oscillator
PH0	—	RCC_OSC_IN	HSE 8 MHz oscillator
PH1	—	RCC_OSC_OUT	HSE 8 MHz oscillator

4.4 Timer Configuration

4.4.1 TIM1 — PWM Generation (motor control)

TIM1 is the advanced APB2 timer (clock 168 MHz) configured in **PWM Generation** mode on 4 simultaneous channels.

Table 4.4: TIM1 parameters — Motor PWM

Parameter	Value
TIM1 clock	168 MHz ($APB2 \times 2$)
Prescaler (PSC)	0 (no division)
Auto-Reload (ARR)	9999
PWM frequency	$168 \text{ MHz} / (0 + 1) / (9999 + 1) = \mathbf{16.8 \text{ kHz}}$
PWM resolution	10000 levels (0–9999)
Channels	CH1 (PA8), CH2 (PA9), CH3 (PA10), CH4 (PA11)
Auto-Reload Preload	Enabled

Note

The frequency of 16.8 kHz is above the 20 kHz audible threshold for most DC motors, avoiding annoying acoustic whistling. The $ARR = 9999$ provides **10000 PWM resolution levels**, i.e. a precision of 0.01%.

4.4.2 TIM2 — Quadrature encoder (left motor)

Table 4.5: TIM2 parameters — Left encoder

Parameter	Value
Mode	Encoder_Interface (TIM_ENCODERMODE_TI12)
Period (ARR)	65535 (16-bit counter)
Phase A	PA0 (TIM2_CH1)
Phase B	PA1 (TIM2_CH2)
Auto-Reload Preload	Enabled
Counting	Up and down on TI1 and TI2

4.4.3 TIM3 — Quadrature encoder (right motor)

Table 4.6: TIM3 parameters — Right encoder

Parameter	Value
Mode	Encoder_Interface (TIM_ENCODERMODE_TI12)
Phase A	PA6 (TIM3_CH1)
Phase B	PA7 (TIM3_CH2)
IC1 filter	15 (maximum debounce)
Auto-Reload Preload	Enabled

4.4.4 TIM5 — μ s time measurement (HC-SR04)

Table 4.7: TIM5 parameters — μ s counter

Parameter	Value
Clock source	Internal ($APB1 \times 2 = 84 \text{ MHz}$)
Prescaler (PSC)	167
Resolution	$84 \text{ MHz} / (167 + 1) = 1 \mu\text{s}/\text{tick}$
Period (ARR)	65535 (overflow after 65.5 ms)
Auto-Reload Preload	Enabled

4.5 I2C1 Configuration — MPU-6050

Table 4.8: I2C1 parameters — MPU-6050

Parameter	Value
Mode	I2C Fast (400 kHz)
SDA	PB9
SCL	PB8
MPU-6050 address	0x68 (AD0 $\rightarrow$ GND) or 0x69 (AD0 $\rightarrow$ VCC)

4.6 USART2 Configuration

Table 4.9: USART2 parameters

Parameter	Value
Mode	Asynchronous
TX	PA2
RX	PA3
Baud rate (recommended)	115200 baud
Format	8N1

Chapter 5

Software Architecture

5.1 STM32CubeIDE Project Structure

The project generated by STM32CubeMX `maze_test` follows the standard HAL structure:

```
maze_test/  
  Core/  
    Inc/  
      main.h  
      stm32f4xx_hal_conf.h  
      stm32f4xx_it.h  
    Src/  
      main.c           ← Entry point + navigation logic  
      stm32f4xx_hal_msp.c ← HAL peripheral init  
      stm32f4xx_it.c   ← ISR (SysTick, NVIC)  
      syscalls.c  
  Drivers/  
    CMSIS/  
      STM32F4xx_HAL_Driver/  
maze_test.ioc           ← CubeMX configuration  
STM32F446RETX_FLASH.ld ← Flash linker script  
STM32F446RETX_RAM.ld   ← RAM linker script
```

5.2 Recommended Software Modules

Table 5.1: Modular organization of the STM32 firmware

File	Dependency	Role
<code>main.c</code>	HAL	Entry point, main loop, Flood Fill
<code>motors.h</code>	TIM1 (PWM)	PWM init, <code>drive()</code> , <code>stop()</code>
<code>encoders.h</code>	TIM2, TIM3 (QEI)	Counter reading, distance calculation
<code>hcsr04.h</code>	TIM5 (μ s), GPIO	Ultrasonic distance measurement
<code>mpu6050.h</code>	I2C1	Gyroscope reading, calibration
<code>movement.h</code>	motors + encoders	<code>move_forward()</code> , <code>turn_left/right()</code>
<code>flood_fill.h</code>	—	BFS, <code>updateWalls()</code> , <code>moveToNextCell()</code>
<code>debug.h</code>	USART2	<code>printf</code> redirected to USART2

5.3 Main Flow Diagram

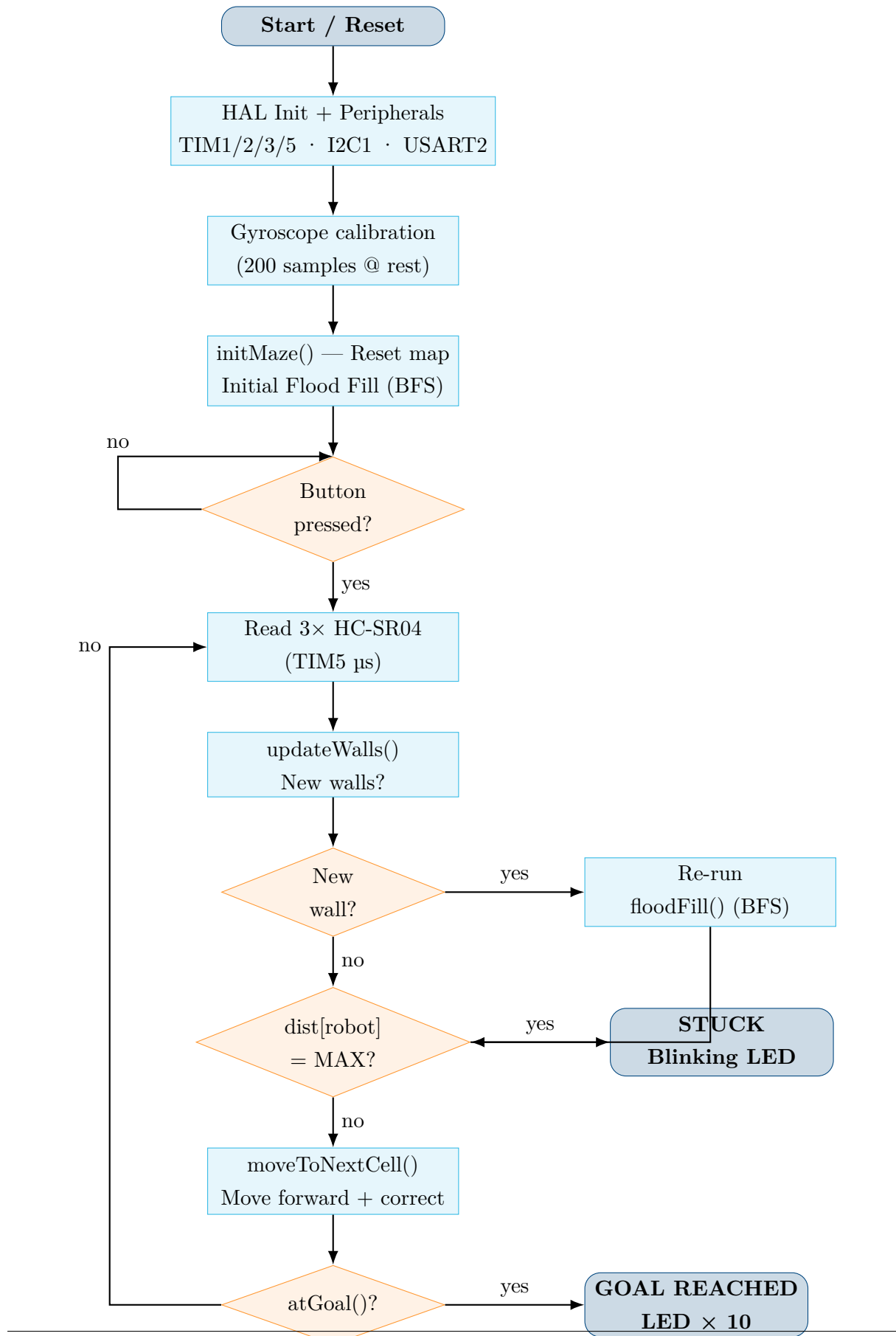


Figure 5.1: Flow diagram — main navigation loop

Chapter 6

Hardware Drivers (HAL STM32)

6.1 HC-SR04 Driver — Ultrasonic Measurement

6.1.1 Operating Principle

1. Send a $10\mu\text{s}$ TRIG pulse (high level)
2. Wait for the rising edge of the ECHO signal
3. Start TIM5 ($1\mu\text{s}$ resolution)
4. Wait for the falling edge of ECHO
5. Stop TIM5 — read counter Δt
6. Distance: $d = \Delta t \times 0.0343/2$ [cm]

6.1.2 HAL Code — Distance Measurement

```
1  /* hcsr04.h      Ultrasonic distance measurement via TIM5 (1 us/
   tick) */
2  #define SPEED_OF_SOUND_CM_US  0.0343f
3  #define HC_TIMEOUT_US          25000  /* 25 ms max (4 m) */
4
5  typedef struct {
6      GPIO_TypeDef *trig_port;
7      uint16_t      trig_pin;
8      GPIO_TypeDef *echo_port;
9      uint16_t      echo_pin;
10 } HCSR04_t;
11
12 /* Reset TIM5 counter */
```

```

13 static inline void TIM5_reset(void) {
14     __HAL_TIM_SET_COUNTER(&htim5, 0);
15 }
16
17 /* Return TIM5 counter in s */
18 static inline uint32_t TIM5_read_us(void) {
19     return __HAL_TIM_GET_COUNTER(&htim5);
20 }
21
22 float HCSR04_measure_cm(HCSR04_t *sensor) {
23     uint32_t t_start, t_end, elapsed;
24
25     /* --- 1. TRIG pulse 10 s --- */
26     HAL_GPIO_WritePin(sensor->trig_port, sensor->trig_pin,
27         GPIO_PIN_SET);
28     TIM5_reset();
29     while (TIM5_read_us() < 10); /* wait 10 s */
30     HAL_GPIO_WritePin(sensor->trig_port, sensor->trig_pin,
31         GPIO_PIN_RESET);
32
33     /* --- 2. Wait for ECHO rising edge --- */
34     TIM5_reset();
35     while (!HAL_GPIO_ReadPin(sensor->echo_port, sensor->echo_pin)
36         ) {
37         if (TIM5_read_us() > HC_TIMEOUT_US) return -1.0f; /*
38             timeout */
39     }
40
41     /* --- 3. Measure ECHO high duration --- */
42     t_start = TIM5_read_us();
43     while (HAL_GPIO_ReadPin(sensor->echo_port, sensor->echo_pin))
44     {
45         if ((TIM5_read_us() - t_start) > HC_TIMEOUT_US) return
46             -1.0f;
47     }
48     t_end = TIM5_read_us();
49     elapsed = t_end - t_start; /* duration in s */
50
51     return (float)elapsed * SPEED_OF_SOUND_CM_US / 2.0f;
52 }

```

Listing 6.1: HC-SR04 Driver — STM32 HAL

6.1.3 HC-SR04 Instances

```

1  /* Instances of the three sensors */
2  HCSR04_t sensor_front = {
3      .trig_port = GPIOB, .trig_pin = GPIO_PIN_0,    /* TRIG_FRONT
4          */
5      .echo_port = GPIOB, .echo_pin = GPIO_PIN_1    /* ECHO_FRONT
6          */
7  };
8  HCSR04_t sensor_left = {
9      .trig_port = GPIOB, .trig_pin = GPIO_PIN_2,    /* TRIG_LEFT
10         */
11     .echo_port = GPIOB, .echo_pin = GPIO_PIN_10    /* ECHO_LEFT
12         */
13 };
14 HCSR04_t sensor_right = {
15     .trig_port = GPIOB, .trig_pin = GPIO_PIN_12,    /* TRIG_RIGHT
16         */
17     .echo_port = GPIOB, .echo_pin = GPIO_PIN_13    /* ECHO_RIGHT
18         */
19 };

```

Listing 6.2: Initialization of HC-SR04 sensors

6.2 BTS7960B Driver — PWM Motor Control

6.2.1 Principle

Each BTS7960B is a high-power half H-bridge (43 A). Two chips form a full H-bridge per motor. Control is achieved via:

- **PWM_R / PWM_L**: PWM signals (TIM1) for speed
- **R_EN / L_EN**: enable (logic level)

6.2.2 HAL Code — PWM Control

```

1  /* Start all 4 TIM1 PWM channels */
2  void motors_init(void) {
3      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1); /* PA8 - Left motor
4          forward */
5      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2); /* PA9 - Left motor
6          backward */
7  }

```



```

5     HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3); /* PA10 - Right
        motor forward */
6     HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4); /* PA11 - Right
        motor backward */
7 }
8
9 /* speed: 0 to 9999 (ARR = 9999) */
10 void motor_set_speed(int16_t speed_left, int16_t speed_right) {
11     if (speed_left >= 0) {
12         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, speed_left);
13         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 0);
14     } else {
15         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
16         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, -speed_left)
            ;
17     }
18     if (speed_right >= 0) {
19         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, speed_right)
            ;
20         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 0);
21     } else {
22         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 0);
23         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, -speed_right
            );
24     }
25 }
26
27 void motors_stop(void) {
28     motor_set_speed(0, 0);
29 }

```

Listing 6.3: Motor control via TIM1 PWM

6.3 MPU-6050 Driver — I2C Gyroscope

6.3.1 Reading the Z Axis

```

1 #define MPU6050_ADDR          (0x68 << 1)
2 #define MPU6050_PWR_MGMT_1    0x6B
3 #define MPU6050_GYRO_ZOUT_H    0x47
4 #define GYRO_SCALE              131.0f /* LSB / deg/s ( 250 deg/s)
        */

```

```

5
6 void MPU6050_init(void) {
7     uint8_t data = 0x00; /* Wake up (clear sleep bit) */
8     HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, MPU6050_PWR_MGMT_1,
9                       1, &data, 1, 100);
10 }
11
12 float MPU6050_read_gyro_z(void) {
13     uint8_t buf[2];
14     HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, MPU6050_GYRO_ZOUT_H,
15                     1, buf, 2, 100);
16     int16_t raw = (int16_t)((buf[0] << 8) | buf[1]);
17     return (float)raw / GYRO_SCALE; /* deg/s */
18 }
19
20 /* Calibration: average of 200 samples at rest */
21 float gyro_bias_z = 0.0f;
22 void MPU6050_calibrate(void) {
23     float sum = 0.0f;
24     for (int i = 0; i < 200; i++) {
25         sum += MPU6050_read_gyro_z();
26         HAL_Delay(5);
27     }
28     gyro_bias_z = sum / 200.0f;
29 }

```

Listing 6.4: MPU-6050 reading via I2C1

6.4 Quadrature Encoder Reading

```

1 /* Read and reset encoder counters */
2 typedef struct {
3     int32_t left;
4     int32_t right;
5 } EncoderCounts_t;
6
7 EncoderCounts_t encoders_read_reset(void) {
8     EncoderCounts_t counts;
9     /* Read 16-bit counters with overflow handling */
10    counts.left  = (int16_t)__HAL_TIM_GET_COUNTER(&htim2);
11    counts.right = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
12    /* Reset to zero */

```

```
13     __HAL_TIM_SET_COUNTER(&htim2, 0);  
14     __HAL_TIM_SET_COUNTER(&htim3, 0);  
15     return counts;  
16 }
```

Listing 6.5: Encoder reading TIM2/TIM3

Chapter 7

Navigation Algorithm — Flood Fill

7.1 Dynamic Flood Fill Principle

The BFS (*Breadth-First Search*) algorithm assigns each cell a **distance** representing the minimum number of steps to reach the goal (goal cell = distance 0). The robot always moves to the accessible neighbor with the lowest distance. When a new wall is discovered, the full BFS is immediately re-run.

7.2 Map Representation

```
1 #define MAZE_SIZE      5          /* 5x5 cell grid */
2 #define GOAL_X         2
3 #define GOAL_Y         2
4 #define DIST_MAX       255
5
6 /* Wall bitmasks (per cell) */
7 #define WALL_N  0x01
8 #define WALL_E  0x02
9 #define WALL_S  0x04
10 #define WALL_W  0x08
11
12 uint8_t walls[MAZE_SIZE][MAZE_SIZE]; /* Known walls */
13 uint8_t dist [MAZE_SIZE][MAZE_SIZE]; /* BFS distances */
14
15 int robotX = 0, robotY = 0;          /* Virtual position */
16 int robotHeading = 0;                /* 0=N 1=E 2=S 3=W */
17
18 const int DIR_DX[4] = { 0, 1,  0, -1}; /* N E S W */
19 const int DIR_DY[4] = { 1, 0, -1,  0};
```

Listing 7.1: Data structures — Flood Fill

7.3 BFS Implementation

```

1 typedef struct { uint8_t x, y; } QCell;
2 static QCell _bfsQ[MAZE_SIZE * MAZE_SIZE];
3
4 void floodFill(void) {
5     memset(dist, DIST_MAX, sizeof(dist));
6     int head = 0, tail = 0;
7
8     dist[GOAL_Y][GOAL_X] = 0;
9     _bfsQ[tail++] = (QCell){GOAL_X, GOAL_Y};
10
11     while (head < tail) {
12         QCell cur = _bfsQ[head++];
13         uint8_t nd = dist[cur.y][cur.x] + 1;
14         for (int d = 0; d < 4; d++) {
15             if (walls[cur.y][cur.x] & (1 << d)) continue; /* wall
16                                                         */
17             int nx = cur.x + DIR_DX[d];
18             int ny = cur.y + DIR_DY[d];
19             if (nx < 0 || nx >= MAZE_SIZE) continue;
20             if (ny < 0 || ny >= MAZE_SIZE) continue;
21             if (dist[ny][nx] <= nd) continue;
22             dist[ny][nx] = nd;
23             _bfsQ[tail++] = (QCell){nx, ny};
24         }
25     }

```

Listing 7.2: floodFill() — BFS from the goal

7.4 Wall Update

```

1 #define WALL_DISTANCE_CM 20.0f /* Detection threshold: 20 cm */
2
3 int updateWalls(void) {
4     int new_wall = 0;
5

```

```
6   float df = HCSR04_measure_cm(&sensor_front);
7   float dl = HCSR04_measure_cm(&sensor_left);
8   float dr = HCSR04_measure_cm(&sensor_right);
9
10  /* Convert relative directions      absolute based on robot
11     heading */
12  int dir_front = robotHeading;
13  int dir_left  = (robotHeading + 3) & 3;
14  int dir_right = (robotHeading + 1) & 3;
15
16  /* Record walls symmetrically */
17  if (df > 0 && df < WALL_DISTANCE_CM) {
18      if (!(walls[robotY][robotX] & (1 << dir_front))) {
19          walls[robotY][robotX] |= (1 << dir_front);
20          /* Also set wall on neighbor side */
21          int nx = robotX + DIR_DX[dir_front];
22          int ny = robotY + DIR_DY[dir_front];
23          if (nx >= 0 && nx < MAZE_SIZE && ny >= 0 && ny <
24              MAZE_SIZE)
25              walls[ny][nx] |= (1 << ((dir_front + 2) & 3));
26          new_wall = 1;
27      }
28  }
29  /* Repeat for left and right... */
30  return new_wall;
31 }
```

Listing 7.3: updateWalls() — HC-SR04 sensor integration

Chapter 8

Control Loop and Trajectory Correction

8.1 Dual Closed-Loop Control

The `move_forward()` function implements a two-level control loop:

Table 8.1: Correction levels in `move_forward()`

Source	Variable	Calculation and role
Encoders	<code>enc_corr</code>	$(e_L - e_R) \times K_{enc}$, clamped to ± 40 Corrects instantaneous speed imbalance
Gyroscope	<code>gyro_corr</code>	$\int \omega_z dt \times K_{gyro}$, clamped to ± 25 Corrects accumulated angular drift
Final command		<code>motor_set_speed(speed + corr, speed - corr - MOTOR_BIAS)</code>

$$\text{correction} = K_{enc}(e_L - e_R) + K_{gyro} \int_{t_0}^t \omega_z(\tau) d\tau \quad (8.1)$$

8.2 Gyroscopic Turn — 3 Phases

1. **Fast phase:** speed `SPEED_TURN` until $\theta_{target} - 12^\circ$
2. **Slow phase:** reduced speed (PWM 110) over the remaining 12°
3. **Stop + measure:** brake at 80 PWM for 55 ms, then gyroscope integration for 1 s to measure the actual angle

8.3 Configuration Parameters

Table 8.2: Critical parameters — config.h

Parameter	Value	Unit	Description
MAZE_SIZE	5	cells	$n \times n$ grid
PULSES_PER_CELL	to calibrate	pulses	Encoder pulses / cell (400 mm)
PULSES_90	to calibrate	pulses	Pulses for exact 90° turn
SPEED_NORMAL	6000	/9999 PWM	Cruise speed
SPEED_TURN	5000	/9999 PWM	Turning speed
MOTOR_BIAS	to measure	/9999 PWM	Left/right drift correction
WALL_DISTANCE_CM	20.0	cm	HC-SR04 wall detection threshold
GYRO_SCALE	131.0	LSB/°/s	MPU-6050 sensitivity ($\pm 250^\circ/\text{s}$)
TURN_SLOW_DEG	12	degrees	Start of slow turn phase
BRAKE_PWM	3000	/9999 PWM	Braking power
BRAKE_MS	55	ms	Braking duration

Chapter 9

Commissioning Procedure and Tests

9.1 Commissioning Order

Warning

The commissioning order below must be strictly followed. **PULSES_PER_CELL** and **PULSES_90** must be recalibrated every time the surface or battery changes.

Table 9.1: Complete commissioning procedure

Step	Test	Procedure and verification
1	TEST_MOTORS	Activate each motor for 1 s. If a wheel spins backwards, reverse its wires on the BTS7960B
2	TEST_ENCODERS	Display L/R counters. Turn each wheel by hand: counters must increment
3	TEST_SENSORS	Display distances and walls every 200 ms. Place an obstacle and verify detection
4	TEST_MOVE	Move forward one cell. Compare actual distance to 400 mm. Adjust PULSES_PER_CELL
5	TEST_SPIN	Perform $4 \times 90^\circ$. Adjust PULSES_90 until returning to the initial heading ($\pm 2^\circ$)
6	TEST_SQUARE	Square test: $4 \times (\text{move forward} + \text{turn left})$. The robot must return to its starting point
7	NAVIGATION	All flags to 0 $\rightarrow$ Maze mode $\rightarrow$ Press the button

9.2 Troubleshooting Guide

Table 9.2: Troubleshooting table

Symptom	Probable cause	Solution
Robot drifts sideways	Motor bias	Adjust <code>MOTOR_BIAS</code>
Does not travel 400 mm	Incorrect <code>PULSES_PER_CELL</code>	Re-run <code>TEST_MOVE</code>
Turn $\neq 90^\circ$	Incorrect <code>PULSES_90</code>	Re-run <code>TEST_SPIN</code>
HC-SR04 always 0	TIM5 not started	Check <code>HAL_TIM_Base_Start</code>
HC-SR04 unstable	No resistor divider on Echo	Add 1k+2k voltage divider
MPU-6050 KO	Wrong I2C address	Check AD0 pin state
Gyro turns in reverse	Incorrect gyro sign	Invert <code>GYRO_SIGN</code>
Robot STUCK	Wall not detected	Decrease <code>WALL_DISTANCE_CM</code>
Encoders not counting	IC filter too strong	Check <code>IC1Filter</code> TIM3

Chapter 10

Linker Scripts and Memory

10.1 Memory Map

Table 10.1: Memory map of the STM32F446RETx

Region	Address	Size	Content
FLASH	0x08000000	512 kB	Code + constant data
RAM	0x20000000	128 kB	Variables + stack + heap
CCMRAM	—	—	Not available (F446)
Stack	Upper RAM	0x400 (1 kB)	Defined in linker
Heap	RAM	0x200 (512 B)	Defined in linker

The project provides two linker scripts:

- `STM32F446RETX_FLASH.ld` — execution from Flash (production)
- `STM32F446RETX_RAM.ld` — execution from RAM (fast debug)

Chapter 11

Conclusion

11.1 Technical Summary

The **Maze Robot STM32F446RE** constitutes a complete and robust implementation of an autonomous maze solver on a professional embedded platform.

- **Real-time performance:** the Cortex-M4 core @ 168 MHz with FPU guarantees smooth processing of control loops and the BFS algorithm
- **Odometric precision:** hardware quadrature encoders (TIM2/TIM3) provide reliable displacement measurement without CPU load
- **High-resolution PWM:** TIM1 @ 16.8 kHz with 10000 levels enables precise and acoustically silent motor control
- **Precise μ s measurement:** TIM5 with 1 μ s resolution ensures reliable HC-SR04 time-of-flight measurement
- **HAL architecture:** the STM32Cube HAL abstraction layer facilitates portability to other STM32 MCUs
- **SWD debugging:** the native SWD/SWO connection of the NUCLEO-F446RE enables real-time debugging and performance profiling

11.2 Future Improvements

- Addition of a 1-D Kalman filter for HC-SR04 measurement smoothing
- Implementation of Speed Run mode (chaining straight-line cells)
- Monitoring interface via USART2 + serial terminal (Putty/STM32CubeMonitor)
- Migration to FreeRTOS for a multi-task architecture (sensors / navigation / debug)

- Replacement of HC-SR04 with VL53L0X I2C sensors for improved precision

Bibliography

- [1] STMicroelectronics, *STM32F446xC/E Reference Manual (RM0390)*, Rev 6, 2019. https://www.st.com/resource/en/reference_manual/rm0390-stm32f446xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] STMicroelectronics, *STM32F446RE Datasheet (DS10693)*, Rev 8, 2021.
- [3] STMicroelectronics, *STM32CubeF4 HAL and Low-Layer Drivers User Manual (UM1725)*, 2023.
- [4] Elecfreaks, *HC-SR04 Ultrasonic Ranging Module Datasheet*, 2013.
- [5] InvenSense, *MPU-6000/MPU-6050 Product Specification Revision 3.4*, 2013.
- [6] Infineon Technologies, *BTS7960B Industrial H-Bridge Datasheet*, Rev 2.0, 2014.
- [7] Micromouse Online, *Flood Fill Algorithm for Maze Solving*, <https://www.micromouseonline.com>.
- [8] A. Benkridis & M. Moujbeni, *Technical Report Maze Robot — ESP32 Flood Fill Version 3.0*, April 2026.